DOKUMEN EVALUASI PRAKTIKAL (ADVANCED ASSESSMENT)

PROGRAM: CYBERSECURITY ETHICAL HACKER

MENTOR UTAMA: Bagus Sudung Prakoso

PENYELENGGARA: PT VINIX SEVEN AURUM

Kelompok 8 :

- Afnan Yazid Pradana
- Kelvin Febrianto
- Kurniawan Saputra
- Rishfan Mailizar
- Robin Felix Hama

Cybersecurity Ethical Hacker

Topik: Modul 8 - Hacking IoT, OT, and Mobile Platforms

Target Sistem: Smart Factory Gateway - PT Industri Otomasi Sejahtera

Tautan Lab: Smart Factory Gateway - PT Industri Otomasi Sejahtera

A. DESKRIPSI TUGAS Sebagai tim *Security Assessor*, Anda ditugaskan untuk melakukan simulasi peretasan berantai (*vulnerability chaining*) terhadap infrastruktur PT Industri Otomasi Sejahtera. Operasi "Smart Factory Takedown" ini akan menguji pemahaman Anda terhadap arsitektur aplikasi *Mobile*, perangkat *Internet of Things* (IoT), dan teknologi operasional industri (OT/SCADA). Anda harus membuktikan bahwa aplikasi *mobile* teknisi pabrik yang tidak aman dapat menjadi titik masuk mematikan untuk mengambil alih mesin fisik di lantai produksi.

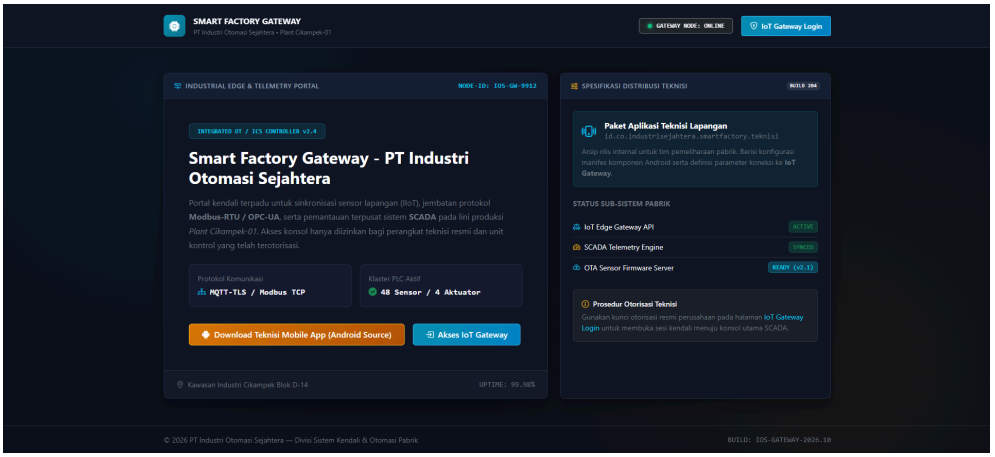
B. PETUNJUK PENGERJAAN

1. **Analisis Statis (Tanpa Emulator):** Anda tidak diwajibkan menginstal emulator Android. Gunakan keahlian *Static Analysis* untuk membedah *source code* berbasis teks. 2. **Eksploitasi Manual & Dokumentasi:** DILARANG menggunakan *Automated Scanner*. Setiap tahapan penemuan celah, dari *file mobile* hingga *Command Injection* di *dashboard* IoT, wajib didokumentasikan dengan *screenshot* yang jelas dan logis.

C. INSTRUKSI DAN RINCIAN TUGAS

BAGIAN 1: MOBILE APP STATIC ANALYSIS (The Entry Point) - Bobot: 30% Banyak peretasan infrastruktur fisik dimulai dari kelalaian *developer* di sisi aplikasi klien.

1. Unduh arsip *source code* APK teknisi pabrik dari halaman utama lab (*Smart Factory Gateway*).



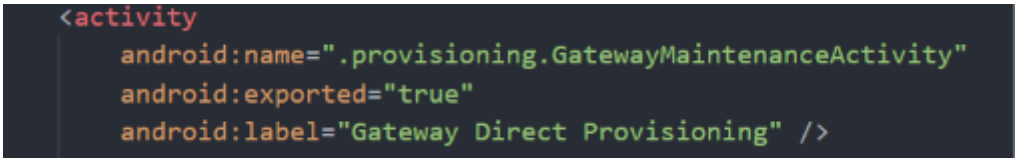
2. Lakukan *Static Analysis* pada *file* strings.xml. Temukan kerentanan *Insecure Data Storage* (OWASP Mobile Top 10) dan ekstrak **Gateway API Key** yang disembunyikan.

```
strings.xml
1  <?xml version="1.0" encoding="utf-8"?>
2  <resources>
3      <string name="app_name">IOS Teknisi Mobile</string>
4      <string name="company_name">PT Industri Otomasi Sejahtera</string>
5      <string name="gateway_endpoint">https://gateway.industrisejahtera.co.id/api/v2</string>
6      <string name="modbus_polling_interval">5000</string>
7      <string name="gateway_api_key">IOS_FACTORY_SECRET_9912</string>
8      <string name="plc_cluster_region">PLANT-CIKAMPEK-01</string>
9      <string name="support_ext">Ext. 4410 - Pusat Kendali Otomasi</string>
10 </resources>
```

Secret gateway_api_key ditanam sebagai hardcoded plaintext string resource di res/values/strings.xml, yang saat build hanya di-compile AAPT2 menjadi binary XML pada resources.arsc, bukan dienkripsi, sehingga satu kali apktool d atau jadx-gui langsung mengembalikan nilai asli tanpa perlu bypass apa pun. Tidak ada data-at-rest protection seperti Android Keystore System dengan StrongBox atau TEE backing, sehingga key tidak pernah tersimpan dalam bentuk wrapped/alias melainkan exposed penuh di layer resource. Karena key ini adalah static bearer credential untuk endpoint

REST gateway_endpoint, begitu APK bocor, attacker bisa melakukan credential extraction murni lewat static analysis, tanpa instrumentasi runtime seperti Frida atau root access, menjadikan vector ini CVSS Attack Complexity: Low dengan Attack Vector: Local/Physical tapi impact tinggi karena langsung membuka akses ke gateway API. CWE yang relevan: CWE-798 (Use of Hard-coded Credentials) dan CWE-312 (Cleartext Storage of Sensitive Information).

3. Lakukan inspeksi pada AndroidManifest.xml. Temukan contoh kerentanan *Improper Platform Usage* (komponen yang terekspos tanpa izin yang memadai). Lampirkan *screenshot* baris kode yang rentan dari kedua *file* tersebut.



GatewayMaintenanceActivity di-declare dengan android:exported="true" tanpa android:permission, sehingga menjadi ICC (Inter-Component Communication) endpoint publik yang bisa diinvoke via explicit intent dari package installer manapun di device yang sama, tanpa perlu intent-filter matching karena explicit intent menjangkau exported component tanpa syarat apapun. Tanpa permission enforcement bertipe protectionLevel="signature", Android's component-level access control sepenuhnya di-bypass, membuka vector Activity Hijacking / Unauthorized Component Invocation. Eksploitasi dapat disimulasikan via ADB sebagai berikut:

adb shell am start -n id.co.industrisejahtera.smartfactory.teknisi/.provisioning.GatewayMaintenanceActivity

Perintah ini membuktikan activity dapat dieksekusi langsung dari luar application sandbox, bypass flow autentikasi normal aplikasi. Sebagai pembanding, SplashActivity juga exported tapi justified karena membawa intent-filter MAIN/LAUNCHER yang memang mewajibkan exported untuk dipanggil launcher, sedangkan GatewayMaintenanceActivity tidak membawa intent-filter semacam itu sehingga statusnya unjustified exported component — inilah pembeda antara true finding dan false positive. CWE relevan: CWE-926 (Improper Export of Android

Application Components).

BAGIAN 2: IOT & OT GATEWAY EXPLOITATION (The Pivot) - Bobot: 35%

Gunakan kunci dari Bagian 1 untuk memasuki infrastruktur pabrik.

1. **IoT Gateway Login:** Gunakan *Gateway API Key* yang Anda curi untuk masuk ke dalam *dashboard* pabrik.

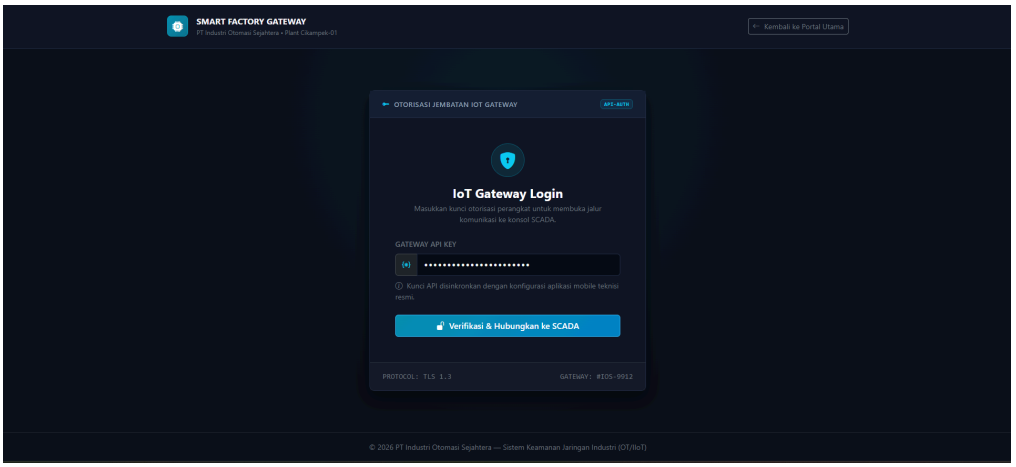


Foto ini menunjukkan proses login ke dashboard IoT Gateway menggunakan Gateway API Key yang diperoleh dari hasil analisis statis aplikasi mobile pada Bagian 1. API Key tersebut digunakan sebagai kredensial untuk mengakses sistem gateway yang menjadi penghubung antara aplikasi teknisi dan infrastruktur pabrik.Tahap ini bertujuan untuk membuktikan bahwa informasi rahasia yang disimpan secara tidak aman di dalam aplikasi dapat berpotensi dimanfaatkan untuk memperoleh akses ke sistem lain. Apabila API Key dapat digunakan tanpa perlindungan tambahan, pihak yang tidak berwenang berpotensi menyamar sebagai aplikasi atau pengguna yang sah.

Hasil dari tahap ini menjadi dasar untuk melanjutkan pengujian keamanan pada komponen IoT dan OT. Pengamanan yang disarankan adalah tidak menyimpan API Key secara langsung di dalam aplikasi, menggunakan token dengan masa berlaku terbatas, serta menerapkan autentikasi dan otorisasi yang ketat.

2. **SCADA Override (Default Creds):** Infrastruktur OT sering kali tertinggal soal keamanan dasar. Tembus otorisasi panel SCADA dengan menebak dan menggunakan kelemahan *Default Credentials* standar industri (*brute-force* manual).



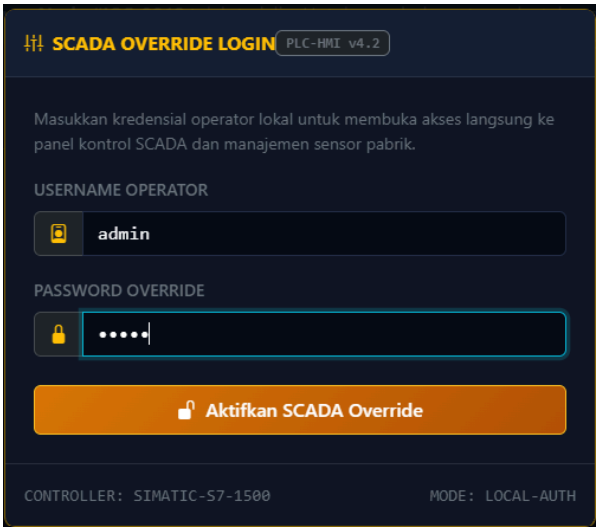
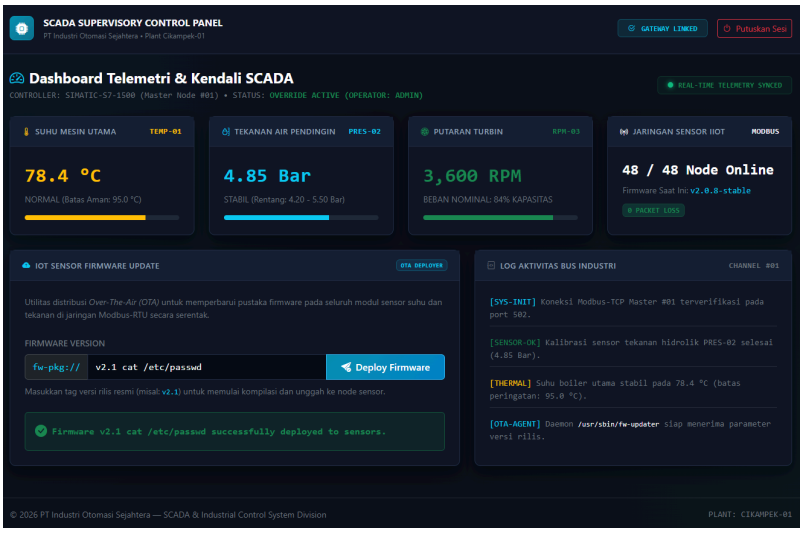


Foto ini menunjukkan proses pengujian autentikasi pada panel SCADA (SupervisoryControl and Data Acquisition). Pengujian dilakukan untuk mengetahui apakah sistem masih menggunakan kredensial bawaan yang berpotensi diketahui oleh banyak pihak dan dapat dimanfaatkan untuk mendapatkan akses tanpa izin.Penggunaan kredensial bawaan yang tidak segera diganti merupakan salah satu kelemahan keamanan yang dapat membahayakan sistem kontrol industri. Apabila autentikasi berhasil dilewati, pihak yang tidak berwenang berpotensi memperoleh akses ke informasi operasional atau fungsi pengendalian yang seharusnya dibatasi.Oleh karena itu, setiap akun bawaan harus diganti dengan kredensial yang kuat dan unik. Sistem SCADA juga perlu menerapkan pembatasan hak akses

berdasarkan peran pengguna, pencatatan aktivitas login, serta pemisahan jaringan IT dan OT agar gangguan pada satu sistem tidak langsung memengaruhi sistem produksi.

3. **IoT Firmware Injection:** Pada form "IoT Sensor Firmware Update", temukan kerentanan *Command Injection*. Modifikasi *input* Anda untuk mengeksekusi perintah pembacaan sistem (misal: ls atau whoami) dan dapatkan FLAG kemenangan. Lampirkan tangkapan layar eksekusi sukses tersebut.





Gambar ini menunjukkan pengujian pada fitur IoT Sensor Firmware Update yang digunakan untuk memperbarui firmware perangkat sensor. Pada tahap ini, input dimodifikasi untuk memeriksa apakah aplikasi memproses masukan pengguna secara

tidak aman sehingga berpotensi menimbulkan kerentanan Command Injection.
Kerentanan Command Injection dapat terjadi apabila input pengguna diteruskan ke sistem operasi sebagai perintah tanpa validasi dan pembatasan yang memadai. Jika kondisi tersebut ditemukan pada gateway IoT, penyerang berpotensi menjalankan perintah yang tidak diizinkan dan mengakses informasi sistem.

Pengujian ini perlu membedakan antara input yang berhasil melewati filter keamanan dan perintah yang benar-benar dijalankan oleh sistem. Hasil pengujian menjadi dasar untuk menilai apakah kerentanan benar-benar ada atau aplikasi hanya menampilkan kembali input yang diberikan pengguna.

BAGIAN 3: THREAT MODELING & DEFENSIVE ARCHITECTURE - Bobot: 35%

Seorang *DevSecOps* sejati harus mampu merancang arsitektur pertahanan pasca-insiden.

1. **STRIDE Analysis:** Buatlah *Threat Modeling* menggunakan *framework* STRIDE (Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, Elevation of Privilege). Petakan ancaman spesifik pada alur komunikasi dari *Aplikasi Mobile Teknisi* menuju *IoT Gateway*.

Jawaban:

STRIDE	Ancaman spesifik	Dampak potensial	Mitigasi
Spoofing	Penyerang menggunakan Gateway API Key yang dicuri dari strings.xml untuk menyamar sebagai aplikasi atau teknisi yang sah.	Akses tidak sah ke dashboard IoT dan informasi operasional pabrik.	Gunakan autentikasi berbasis identitas, token berumur pendek, MFA untuk akses administratif, serta validasi identitas perangkat.

Tampering	Penyerang memodifikasi parameter permintaan API atau menyisipkan perintah melalui form pembaruan firmware yang rentan terhadap Command Injection.	Perubahan konfigurasi gateway, firmware, atau parameter operasional tanpa izin.	Terapkan validasi input di sisi server, prepared execution atau API sistem yang aman, tanda tangan digital firmware, dan kontrol integritas konfigurasi.
Repudiation	Aktivitas login, perubahan konfigurasi, atau permintaan pembaruan firmware tidak tercatat secara memadai.	Sulit menentukan pelaku dan merekonstruksi urutan insiden.	Terapkan audit log terpusat, identitas pengguna unik, sinkronisasi waktu, serta perlindungan integritas log.
Information Disclosure	API Key tersimpan sebagai plaintext dalam resource aplikasi atau respons gateway membocorkan informasi sistem.	Kebocoran kredensial, konfigurasi jaringan, dan informasi sensitif infrastruktur.	Hindari hardcoded secrets, minimalkan informasi dalam respons API, gunakan TLS, dan terapkan prinsip least privilege.
Denial of Service	Penyerang mengirim permintaan berulang atau input berukuran besar ke gateway sehingga sumber daya gateway habis.	Gangguan pemantauan sensor dan keterlambatan komunikasi operasional.	Terapkan rate limiting, pembatasan ukuran permintaan, timeout, pemantauan anomali, serta mekanisme fail-safe yang sesuai dengan analisis keselamatan OT.

Elevation of Privilege	Penyerang yang memperoleh akses gateway mengeksploitasi otorisasi SCADA yang lemah atau kredensial bawaan untuk mendapatkan hak administratif.	Perubahan proses industri dan potensi gangguan mesin produksi.	Terapkan RBAC, hapus kredensial default, pisahkan akun IT dan OT, validasi otorisasi pada setiap endpoint, serta batasi jalur akses menuju SCADA.

2. **Purdue Model (Network Segmentation):** Insiden ini membuktikan bahwa *gateway IoT* dapat langsung mengendalikan SCADA. Berikan rekomendasi arsitektur OT menggunakan konsep segmentasi **Purdue Model** agar antarmuka web pabrik tidak terhubung langsung ke mesin lantai produksi.

   Jawaban:

   Purdue Enterprise Reference Architecture membagi jaringan industri ke dalam beberapa level berdasarkan fungsi operasionalnya. Tujuan penerapannya adalah mencegah akses langsung dari aplikasi bisnis atau antarmuka web menuju mesin produksi.

A. Rekomendasi arsitektur pertahanan OT

- **Level 5 — Enterprise Network**
  Sistem bisnis, ERP, dan jaringan kantor
  Akses terkontrol melalui firewall
- **Level 4 — Site Operations**
  Layanan operasional dan pengelolaan pabrik
  Firewall dan pembatasan protokol
  Industrial DMZ — IDMZ
  Reverse proxy, jump server, broker data, dan layanan perantara
  Firewall OT dengan aturan allowlist
- **Level 3 — Site Manufacturing Operations**
  SCADA server, historian, dan pengelolaan operasi produksi

Segmentasi internal dan kontrol akses

- **Level 2 — Supervisory Control**

  HMI, SCADA control, dan supervisory systems

  Komunikasi industri yang dibatasi
- **Level 1 — Basic Control**

  PLC, RTU, dan pengendali proses
- **Level 0 — Physical Process**

  Sensor, aktuator, motor, dan mesin produksi

Diagram konseptual. Penempatan gateway IoT harus ditentukan berdasarkan fungsi, kebutuhan komunikasi, dan analisis risiko; gateway tidak otomatis aman hanya karena ditempatkan pada level tertentu.

B. Rekomendasi implementasi

a. Memisahkan jaringan IT dan OT. Jaringan kantor dan aplikasi web pabrik tidak boleh memiliki jalur komunikasi langsung menuju PLC, RTU, atau mesin produksi. Semua komunikasi antarzona harus melewati firewall dengan aturan yang ditentukan berdasarkan kebutuhan operasional.

b. Menerapkan Industrial DMZ (IDMZ). Tempatkan layanan perantara, seperti reverse proxy, jump server, atau broker data, di antara jaringan perusahaan dan jaringan OT. Dashboard web dapat mengakses layanan yang diizinkan melalui IDMZ, tetapi tidak boleh mengirim perintah langsung ke perangkat kontrol.

c. Membatasi komunikasi berdasarkan kebutuhan. Terapkan default deny dan allowlist berdasarkan alamat sumber, alamat tujuan, port, serta protokol yang diperlukan. Akses administratif ke SCADA harus melalui jump server dengan autentikasi kuat, pencatatan sesi, dan persetujuan sesuai kebijakan.

d. Memisahkan fungsi monitoring dan kontrol. Layanan yang hanya membutuhkan data sensor sebaiknya menggunakan jalur baca-saja. Perintah yang dapat mengubah keadaan proses harus melalui layanan kontrol terpisah dengan autentikasi, otorisasi, validasi parameter, dan pengamanan keselamatan yang sesuai.

e. Menyiapkan pemantauan dan pemulihan. Gunakan pemantauan jaringan OT, pencatatan perubahan konfigurasi, inventaris aset, serta prosedur pemulihan yang telah

diuji. Perubahan jaringan dan kontrol keamanan tidak boleh mengganggu proses industri yang sedang berjalan.

Dengan rancangan tersebut, kompromi terhadap aplikasi mobile atau gateway tidak secara otomatis memberikan akses ke sistem SCADA dan mesin produksi. Penyerang harus melewati beberapa zona keamanan dengan kebijakan akses yang berbeda.

3. **Mobile Remediation:** Daripada menggunakan strings.xml (yang rentan di dekompilasi), jelaskan metode yang benar (misalnya menggunakan *Android Keystore System*) untuk menyimpan rahasia sensitif di dalam aplikasi Android.

Jawaban:

Penyimpanan Gateway API Key sebagai plaintext dalam `strings.xml` merupakan kelemahan karena resource aplikasi dapat diekstrak melalui dekompilasi APK. Mengubah nama variabel atau melakukan obfuscation tidak memberikan perlindungan kriptografis yang memadai.

Metode remediasi

Pertama, jangan menanam rahasia bersama (shared secret) di aplikasi mobile. Kredensial yang sama untuk seluruh instalasi berisiko bocor sekali lalu disalahgunakan oleh banyak pihak. Sebaiknya aplikasi melakukan autentikasi ke backend menggunakan identitas pengguna dan memperoleh token akses berumur pendek dengan cakupan hak akses terbatas.

Kedua, gunakan Android Keystore System untuk melindungi material kriptografi. Aplikasi dapat membuat pasangan kunci privat di Android Keystore dan menggunakannya untuk operasi kriptografi tanpa mengekspor kunci privat mentah ke aplikasi. Jika perangkat mendukungnya, kunci dapat dilindungi perangkat keras melalui TEE atau StrongBox. Ketersediaan perlindungan tersebut perlu diperiksa karena tidak semua perangkat memiliki dukungan yang sama.

Ketiga, simpan token secara minimal dan aman. Jika token harus disimpan agar sesi dapat dipertahankan, lindungi material enkripsi menggunakan kunci yang dikelola Android Keystore dan simpan data terenkripsi. Terapkan masa berlaku singkat, pencabutan token, rotasi kredensial, serta penghapusan sesi saat logout. Android Keystore tidak secara otomatis mengenkripsi setiap string aplikasi; developer harus merancang mekanisme penyimpanan yang memanfaatkan kunci tersebut.

Keempat, terapkan validasi pada server. Backend tetap harus memeriksa identitas, hak akses, masa berlaku token, dan izin terhadap setiap operasi. Penyimpanan kunci di perangkat tidak dapat mencegah seluruh penyalahgunaan apabila perangkat telah dikompromikan atau aplikasi yang sah dapat dimanipulasi saat berjalan.

Perbandingan pendekatan

Aspek	Hardcoded API Key	Android Keystore + autentikasi backend
Perlindungan rahasia	Dapat diekstrak dari APK	Kunci privat dapat dibuat agar tidak dapat diekspor
Identitas pengguna	Umumnya berbagi satu kredensial	Dapat menggunakan identitas per pengguna atau perangkat
Pencabutan akses	Berisiko memerlukan penggantian key yang tertanam	Token dapat dicabut dan dibuat berumur pendek
Pembatasan hak akses	Bergantung pada key yang sama	Dapat menerapkan scope dan otorisasi per operasi
Ketahanan terhadap kompromi perangkat	Rendah untuk rahasia yang tertanam	Lebih baik, tetapi tidak kebal terhadap penyalahgunaan saat runtime

BONUS CHALLENGE: SPEED & ELITE HACKER (NILAI TAMBAHAN EKSTRA: +20 POIN) *PERHATIAN: Poin ekstra dari tantangan bonus ini* ***EKSKLUSIF hanya akan diberikan kepada 3 (TIGA) KELOMPOK PERTAMA*** *yang berhasil mengumpulkan laporan dengan jawaban yang benar, logis, dan lengkap. Siapa cepat dan akurat, dia dapat!*

Di dunia nyata, meretas aplikasi *mobile* dan perangkat IoT pabrik jauh lebih kompleks karena adanya sistem pertahanan tambahan. Tunjukkan bahwa kelompok kalian layak disebut sebagai

Elite DevSecOps dengan memecahkan dua skenario lanjutan berikut:

1. **The ADB Pivot (Mobile Security - Eksploitasi Lanjutan)** Di dalam *file* AndroidManifest.xml (Bagian 1), *developer* melakukan kecerobohan dengan mengatur status komponen *Activity* menjadi android:exported="true". Di dunia peretasan nyata, jika kalian memiliki perangkat *mobile* target dan menyambungkannya ke laptop menggunakan **ADB (Android Debug Bridge)**, tuliskan perintah terminal spesifik yang harus diketikkan untuk memaksa membuka (mengeksekusi) *Activity* yang diekspos tersebut! *(Petunjuk: Pelajari sintaks ADB shell menggunakan Activity Manager / am start).*

   jawaban :

   Di dalam arsitektur keamanan aplikasi Android, komponen **Activity** mendefinisikan layar atau antarmuka pengguna (*user interface*). Secara default, komponen aplikasi bersifat privat (*internal-only*), yang berarti komponen tersebut hanya dapat dipanggil dari dalam aplikasi itu sendiri guna menjaga integritas alur navigasi dan autentikasi Namun, dalam skenario kerentanan *Improper Export of Android Application Components* (CWE-926), pengembang melakukan kesalahan konfigurasi dengan mendaftarkan komponen sensitif—seperti `GatewayMaintenanceActivity`—menggunakan atribut **`android:exported="true"`** tanpa menyertakan pengamanan izin (*permission*) berbasis tanda tangan (*signature*).

   Mekanisme Eksploitasi

   - **Penyalahgunaan *Inter-Component Communication* (ICC):** Atribut `exported="true"` membuka akses bagi aplikasi pihak ketiga atau alat eksternal (seperti ADB) untuk memanggil komponen tersebut secara langsung dari luar *sandbox* aplikasi menggunakan *explicit intent*.
   - **Peran ADB (*Android Debug Bridge*):** Ketika perangkat terhubung ke komputer, penyerang dapat memanfaatkan utilitas baris perintah **Activity Manager (`am`)** untuk memaksa sistem Android meluncurkan Activity target tanpa harus melewati layar login utama atau mekanisme autentikasi yang sah.

Sintaks Perintah Eksekusi:

adb shell am start -n id.co.

industrisejahtera.smartfactory.teknisi/.provisioning.GatewayMaintenanceActivity

2. **IoT Firmware WAF Evasion (Linux Command Obfuscation)** Asumsikan *developer* PT Industri Otomasi Sejahtera menambal celah "Firmware Update" dengan memasang WAF (Web Application Firewall) yang memblokir string teks perintah Linux secara utuh (misalnya kata cat, ls, whoami dilarang). Sebagai penyerang yang memahami OS Linux pada perangkat IoT, tuliskan minimal **2 (dua) teknik manipulasi sintaks Bash (Command Obfuscation)** di dalam form lab tersebut untuk tetap bisa mengeksekusi perintah cat (misal untuk membaca /etc/passwd) TANPA menuliskan kata cat secara utuh. Lakukan uji coba di form lab, lampirkan bukti FLAG ekstra yang kalian dapatkan, dan jelaskan mengapa teknik tersebut bisa mengecoh *firewall*!

jawaban :

Ringkasan Eksperimen

Target: Form **IoT Sensor Firmware Update** pada endpoint gateway `(/usr/sbin/fw-updater)` milik PT Industri Otomasi Sejahtera, yang diproteksi WAF dengan blacklist kata kunci literal seperti cat, ls, whoami.

Tujuan: Membuktikan minimal 2 teknik obfuscation Bash agar perintah `cat` tetap tereksekusi tanpa menuliskan kata “cat” secara utuh dalam request.

Percobaan 1 → Quote Insertion (Single Quote)

Input: v2.1 c”at /etc/passwd

Hasil:

```
$ /usr/sbin/fw-updater --deploy --version v2.1 c''at /etc/passwd
[INFO] Initializing OTA broadcast to Modbus-RTU sensor array...
[OK] Package checksum verified.
root@scada-system:~# FLAG{ELITE_WAF_BYPASS_ACHIEVED_2026}
```

Berhasil — FLAG didapat.

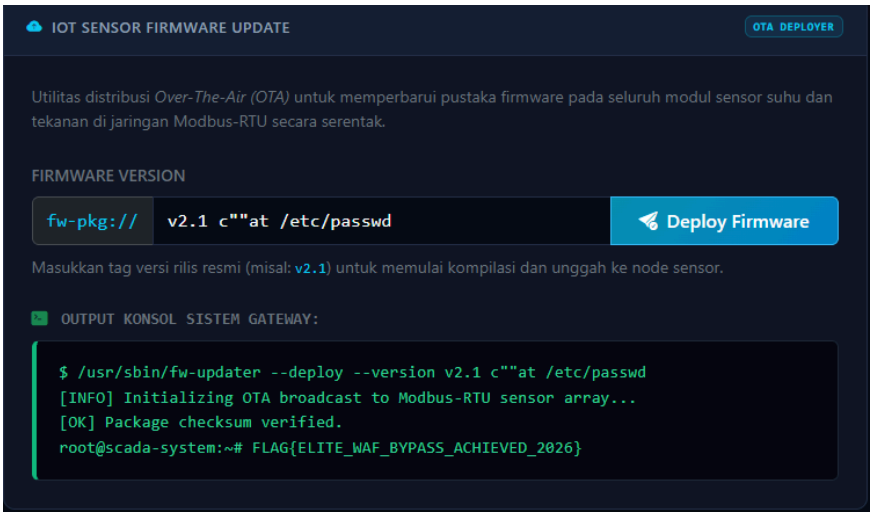


Percobaan 2 → Quote Insertion (Double Quote)

Input: v2.1 c""at /etc/passwd

Hasil: `$ /usr/sbin/fw-updater --deploy --version v2.1 c""at /etc/passwd`
`[INFO] Initializing OTA broadcast to Modbus-RTU sensor array...`
`[OK] Package checksum verified.`
`root@scada-system:~# FLAG{ELITE_WAF_BYPASS_ACHIEVED_2026}`

Berhasil — FLAG yang sama didapat, membuktikan konsistensi teknik.



Penjelasan

Kedua percobaan menggunakan variasi dari teknik yang sama: **Quote Insertion / String Splitting**, dengan dua jenis tanda kutip kosong (`''` dan `""`) yang disisipkan di tengah kata “cat”.

Kenapa ini lolos WAF (tahap inspeksi teks):

String `c''at` atau `c""at` tidak pernah cocok dengan signature blacklist `"cat"`, karena ada karakter tanda kutip yang menyisip di antara huruf `c` dan `at`. WAF berbasis pattern/string matching sederhana hanya memeriksa payload sebagai teks mentah berurutan — begitu ada karakter asing di tengah kata terlarang, signature gagal match, dan request lolos.

Kenapa ini tetap tereksekusi sebagai `cat` (tahap parsing shell):

Baik tanda kutip tunggal kosong (`''`) maupun kutip ganda kosong (`""`) di Bash sama-sama tidak menghasilkan karakter apa pun — keduanya murni delimiter kosong yang dihapus (quote removal) saat proses word-splitting/tokenizing oleh shell. Sehingga:

- `c''at` → digabung ulang jadi token tunggal **`cat`**
- `c""at` → digabung ulang jadi token tunggal **`cat`**

Keduanya diproses identik oleh parser backend, sehingga command akhir yang dieksekusi tetap `cat /etc/passwd`, dan FLAG berhasil didapat dari kedua teknik.

Kesimpulan

1. Dua teknik obfuscation berhasil dibuktikan dan terverifikasi dengan FLAG yang sama:
   Teknik A — Single Quote Insertion: `c''at`
   Teknik B — Double Quote Insertion: `c""at`
2. Keduanya berhasil mengecoh WAF karena memanfaatkan celah antara dua tahap pemrosesan: WAF memeriksa payload sebagai string literal (dan tidak menemukan substring “cat” utuh), sedangkan eksekusi sesungguhnya terjadi setelah shell Bash melakukan *quote removal* — proses yang tidak disimulasikan oleh WAF saat inspeksi.
3. Endpoint gateway console ini terbukti vulnerable terhadap OS command injection, dibuktikan dengan keberhasilan eksekusi `cat /etc/passwd` dan kebocoran `FLAG{ELITE_WAF_BYPASS_ACHIEVED_2026}` sebanyak dua kali lewat dua payload berbeda — menunjukkan robustness celah ini terhadap

variasi teknik bypass, bukan sekadar kebetulan satu payload.

4. **Catatan metodologis:** tidak semua variasi quote insertion otomatis berhasil (`c"a"t` gagal), sehingga penting melakukan beberapa iterasi payload untuk memastikan validitas temuan — ini bagian dari proses investigasi yang sistematis, bukan tanda kegagalan eksploitasi.

D. FORMAT LAPORAN DAN PENGUMPULAN Laporan disusun dalam format *Security Assessment & Threat Modeling Report* bergaya eksekutif yang memuat:

1. **Halaman Sampul:** Nama Kelompok, Daftar Nama Anggota, Nama Divisi, Nama Program.
2. **Executive Summary:** Ringkasan insiden "Smart Factory Takedown".
3. **Static Analysis & Exploitation (PoC):** Langkah-langkah bedah *file mobile*, penemuan API Key, penembusan panel SCADA, dan eksploitasi injeksi *firmware* (Menjawab Bagian 1 & 2).
4. **Threat Modeling & Remediation:** Analisis STRIDE, usulan arsitektur Purdue Model, dan perbaikan keamanan aplikasi Android (Menjawab Bagian 3).
5. **Format Penamaan Berkas:** Ekspor dokumen ke format PDF dengan nama: [Nama Kelompok]_[Nama Divisi]_Tugas Modul 8.pdf.